# Appliance Energy Consumption Prediction Using Deep Learning

## Assessment: Multivariate Time-Series Prediction Using Deep Learning

ABINAYA R

4/1/26

Dataset: Appliance Energy Prediction

## Table of Contents

1. Introduction
2. Data Insights
3. Preprocessing
4. Feature Engineering
5. Model Design
6. Results
7. Model Optimization
8. Challenges and Solutions
9. Conclusion
10. References

# 1. Introduction

Energy consumption prediction is a critical component of modern smart home systems, building energy management, and sustainability planning. Accurate forecasting of appliance energy usage enables proactive load balancing, cost reduction, and informed decision-making around energy efficiency. As residential energy consumption continues to grow globally, the ability to predict short-term appliance usage with precision has significant practical value.

This assessment addresses the problem of predicting household appliance energy consumption using the Appliance Energy Prediction Dataset, a multivariate time-series dataset recorded at 10-minute intervals across a residential building in Belgium between January and May 2016. The dataset contains 19,735 observations across 29 features including indoor and outdoor temperature sensors, humidity readings, atmospheric pressure, wind speed, visibility, and time-based variables.

The target variable is Appliances — the energy consumption of household appliances measured in watt-hours (Wh). This is a regression task on a time-series with strong temporal dependencies, seasonal patterns, and a right-skewed distribution with extreme values.

The approach taken involves six stages: thorough data preprocessing and cleaning, domain-informed feature engineering, autocorrelation analysis to justify temporal feature choices, baseline model development using Linear Regression and Random Forest, deep learning model development using three architectures — LSTM, GRU, and CNN-LSTM — and systematic model optimization using Random Search hyperparameter tuning and dropout rate experimentation. All models are evaluated on a temporally held-out test set using MAE, RMSE, and $R^2$.

# 2. Data Insights

## 2.1 Dataset Overview

==========

INITIAL DATASET INSPECTION

Shape: (19735, 29)

Date range: 2016-01-11 17:00:00 → 2016-05-27 18:00:00

The raw dataset contains 19,735 records and 29 columns spanning January to May 2016. After removing two intentional noise columns (rv1, rv2) the working dataset contains 27 columns. The target variable Appliances has the following distribution:

Basic statistics (target):

| | |
|---|---|
| count | 19735.000000 |
| mean | 97.694958 |
| std | 102.524891 |
| min | 10.000000 |
| 25% | 50.000000 |
| 50% | 60.000000 |
| 75% | 100.000000 |
| max | 1080.000000 |

The large gap between the median (60 Wh) and the maximum (1,080 Wh) signals a strongly right-skewed distribution with extreme upper values requiring treatment before modeling.

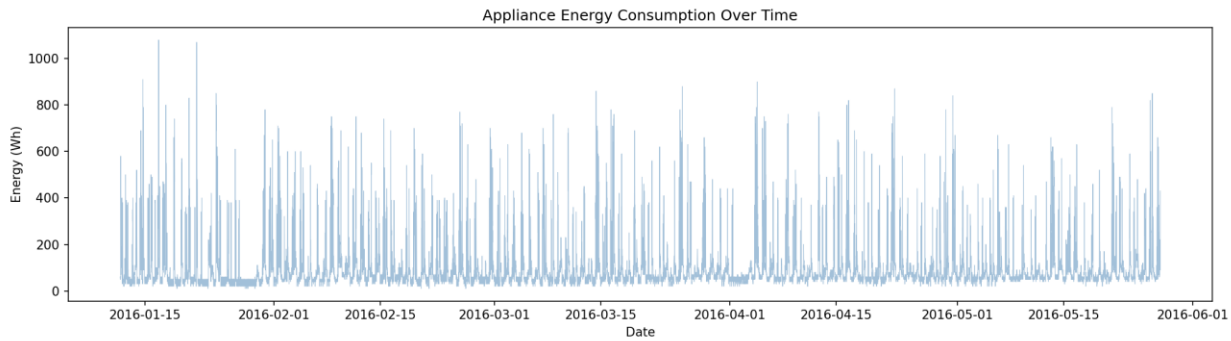


Figure 1: Appliance energy consumption over the full observation period January–May 2016. Clear daily and weekly periodicity is visible alongside irregular high-consumption spikes.

## 2.2 Energy Consumption Over Time

The time-series plot reveals clear daily and weekly periodicity. Short-duration spikes reaching above 500 Wh appear throughout the record, representing real high-consumption events such as oven usage or HVAC activation. The baseline consumption between spikes remains low, consistent with standby and lighting loads. This pattern confirms both regular cyclical structure and irregular event-driven spikes — motivating deep learning architectures capable of capturing both.

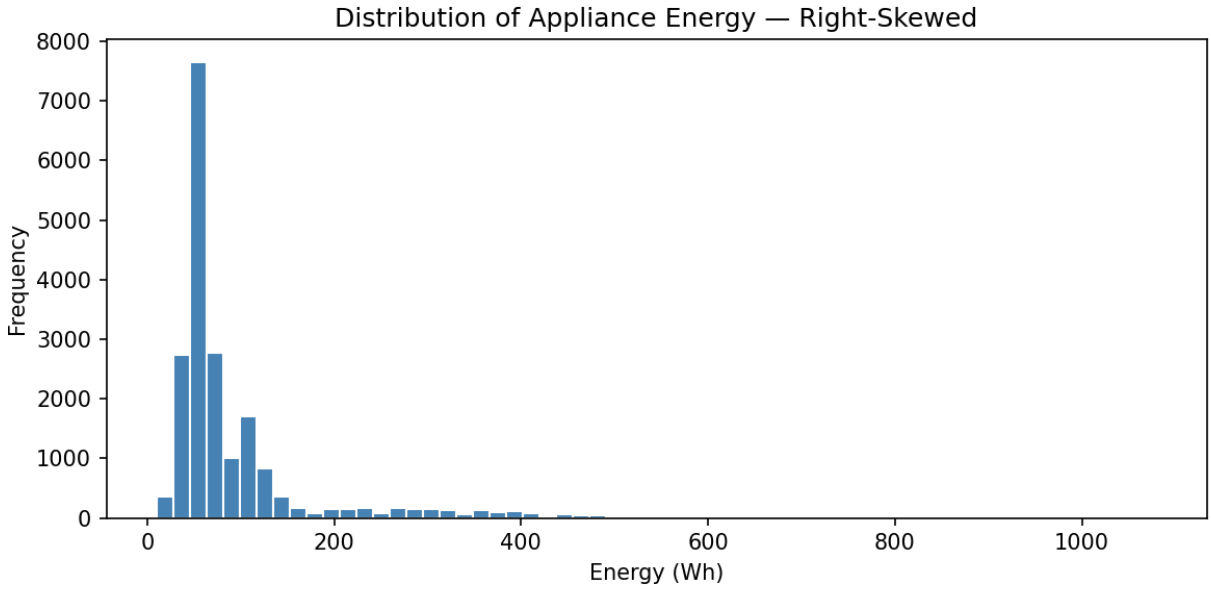


Figure 2: Distribution of appliance energy consumption. Strongly right-skewed with majority of values between 10–200 Wh and a long tail to 1,080 Wh.

## 2.3 Distribution Analysis

The histogram confirms the target variable is strongly right-skewed. The majority of observations cluster between 10 and 200 Wh with a long right tail extending to 1,080 Wh. This has two modeling implications: MSE loss is disproportionately influenced by extreme values during training, and simple mean-based imputation would be inappropriate. These observations directly motivated the choice of Huber loss and 99th percentile capping.

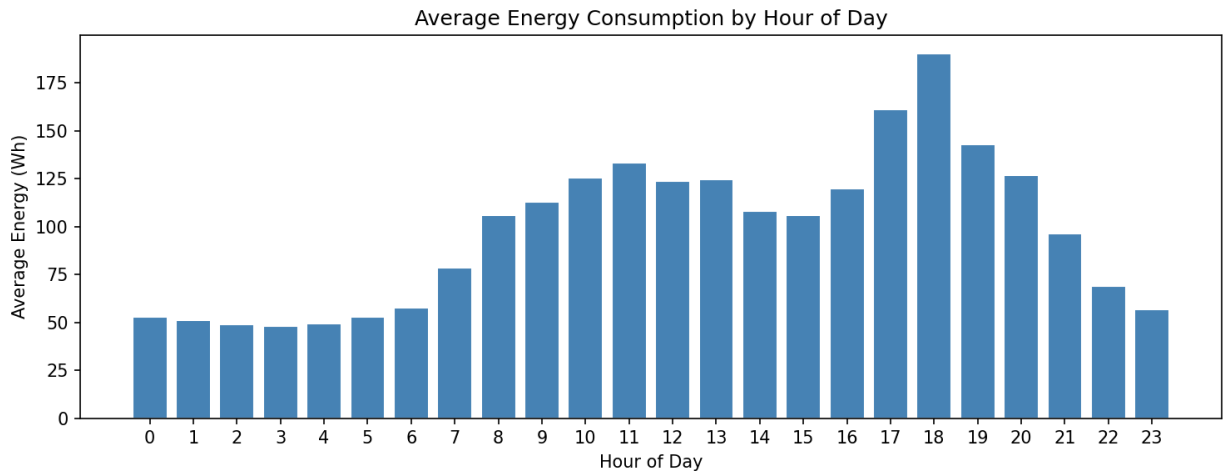


Figure 3: Average energy consumption by hour of day. Morning peak 07:00–09:00 and evening peak 17:00–21:00 reflect typical residential occupancy patterns.

## 2.4 Hourly Consumption Patterns

The bar plot reveals two distinct peaks: morning (07:00–09:00) and evening (17:00–21:00), consistent with typical residential occupancy. Consumption drops to its lowest during overnight hours (01:00–06:00). This clear intra-day pattern directly justifies hour-of-day as a feature and the use of cyclical sine/cosine encoding.

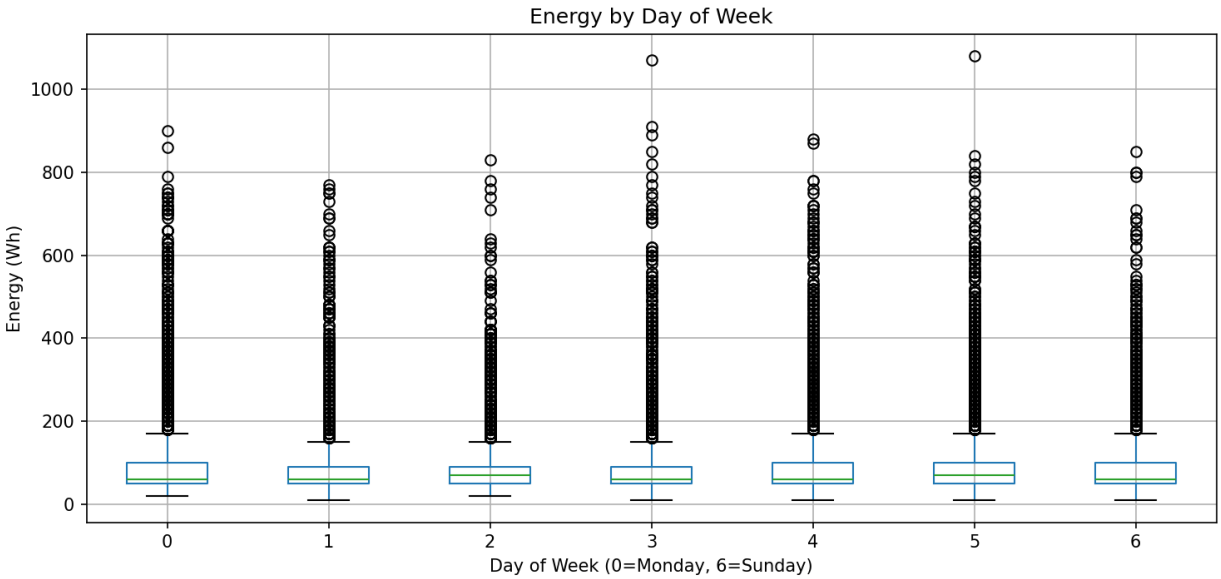


Figure 4: Boxplot of energy consumption by day of week (0=Monday, 6=Sunday). Weekend days show slightly higher median consumption and wider spread.

## 2.5 Day of Week Analysis

Weekend days show slightly different median consumption and wider interquartile ranges compared to weekdays — residents are likely home for more of the day — justifying the inclusion of an is_weekend binary feature.

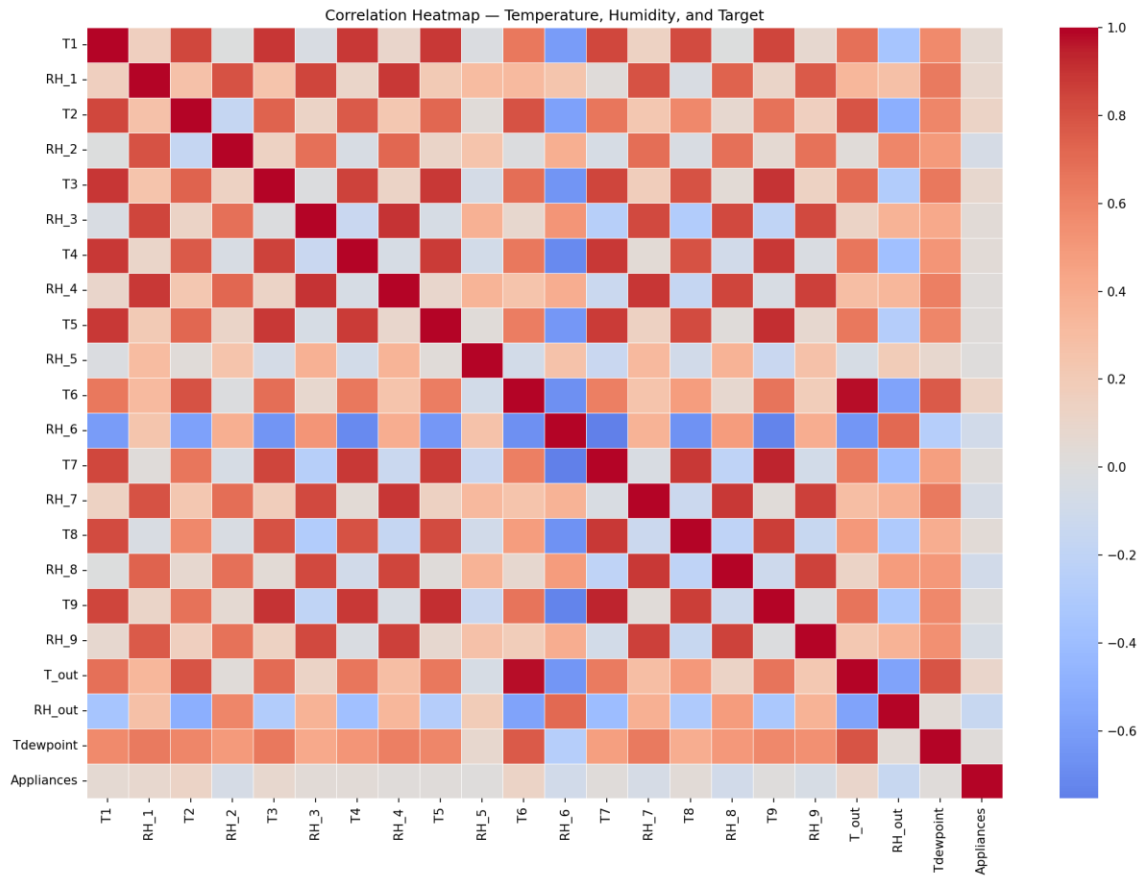


Figure 5: Correlation heatmap of temperature and humidity sensors with target variable. High inter-sensor correlation among T1–T6 motivated creation of avg_indoor_temp.

## 2.6 Correlation Analysis

Indoor temperature sensors show moderate positive correlation with Appliances. The temperature sensors T1 through T6 form a tightly correlated cluster — including all individually would introduce multicollinearity without adding meaningful information. This directly motivated creation of aggregated features avg_indoor_temp and avg_indoor_RH.

# 3.Preprocessing

## 3.1 Data Loading and Sorting

The dataset was loaded with date parsed as datetime and sorted chronologically. Chronological sorting is mandatory — if records are not in temporal order before splitting, future observations can appear in training data, constituting data leakage producing unrealistically optimistic test metrics.

## 3.2 Noise Column Removal

rv1 == rv2 (identical columns): True
rv1 correlation with Appliances: -0.0111
rv2 correlation with Appliances: -0.0111
Shape after dropping noise columns: (19735, 27)

Both rv1 and rv2 were dropped. They are documented noise variables with near-zero correlation to the target. Retaining them would risk the model learning spurious associations.

## 3.3 Missing Value Analysis

```
============================================================
MISSING VALUE ANALYSIS
============================================================
Total missing values: 0

✓ No missing values detected in raw dataset.
  Strategy documented: ffill then bfill will be applied
  to any NaN values arising from lag/rolling operations.
  Rationale: Preserves temporal continuity.
```

**Total missing values detected: 0**

No imputation was required. The strategy is formally documented: forward-fill then backward-fill. Forward-fill propagates the last valid observation forward — physically meaningful for sensor time-series. Mean imputation was explicitly rejected as it introduces a statistical artefact — a flat line at the mean — inconsistent with temporal structure. Features exceeding 50% missing would be dropped entirely.

Note: 36 NaN rows arising from lag feature creation were dropped after feature engineering. Imputing lagged values would introduce false correlations.

## 3.4 Outlier Detection and Treatment

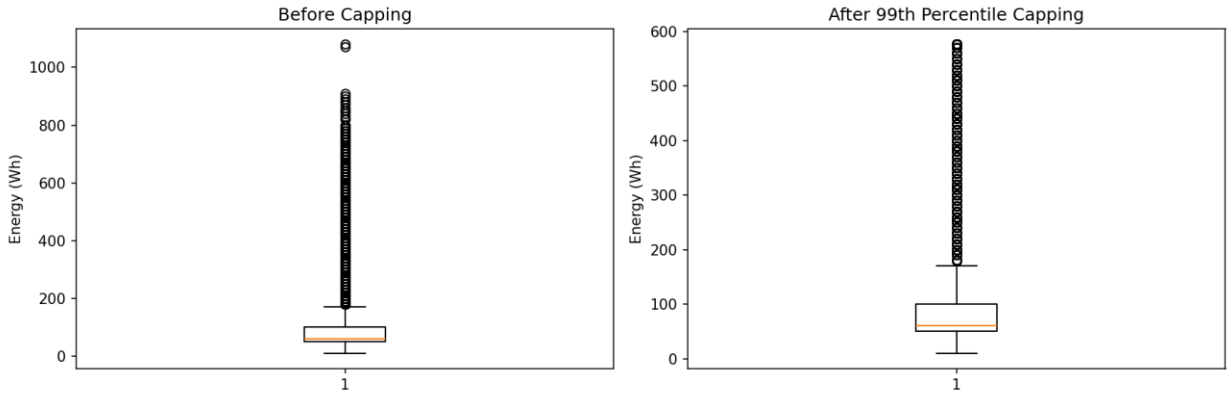


Figure 6: Boxplot before capping (left) and after 99th percentile capping at 576.6 Wh (right). Extreme tail dampened while distribution shape preserved.

IQR bounds: [-25.0, 175.0] Wh
Outliers detected: 2138 (10.83% of data)

99th percentile cap: 576.6 Wh
Records affected: 198 (1.00%)

Conclusion: Capping preserves temporal structure while reducing extreme gradient influence during training.

10.83% of data lies above the IQR upper bound of 175 Wh. These are not measurement errors — they represent real high-consumption events. Removing them would destroy meaningful temporal patterns and reduce training data volume significantly.

Treatment — 99th percentile capping:

| Option | Decision | Reason |
|---|---|---|
| Remove rows | Rejected | Destroys temporal continuity |
| Mean imputation | Rejected | Introduces flat-line artefacts |
| IQR capping at 175 Wh | Rejected | Too aggressive — removes real events |
| **99th percentile cap** | **Selected** | Preserves 99% unchanged; dampens only extreme values |

## 3.5 Data Scaling

Method: Min-Max Scaling [0, 1]

Scaled array shapes:
X_train_raw_sc: (15759, 35) — raw features, train
X_test_raw_sc: (3940, 35) — raw features, test
X_train_eng_sc: (15759, 47) — engineered features, train
X_test_eng_sc: (3940, 47) — engineered features, test
y_train_sc: (15759, 1) — target, train
y_test_sc: (3940, 1) — target, test

Scaler fit range verification:
y min in train: 10.00 → scaled: 0.0000
y max in train: 576.60 → scaled: 1.0000

**Justification:** Neural networks converge faster with bounded inputs. Min-Max preserves the original distribution shape without assuming Gaussian structure. Standardisation was rejected — the right-skewed target does not approximate a normal distribution. All three scalers (scaler_X_raw, scaler_X_eng, scaler_y) were fit exclusively on training data and applied transform-only to the test set to prevent leakage.

## 3.6 Train/Test Split

Train: 15759 rows | 2016-01-11 23:00:00 → 2016-04-30 09:20:00
Test: 3940 rows | 2016-04-30 09:30:00 → 2016-05-27 18:00:00
Split: 80% train / 20% test

The dataset was split chronologically with no shuffling. Shuffling a time series allows future observations into training — a fundamental form of data leakage in temporal prediction.

# 4. Feature Engineering

After preprocessing the dataset contains 27 raw columns. Feature engineering expanded this to 53 columns by creating 23 additional features.

Rows dropped (lag NaN): 36
Shape after feature engineering: (19699, 53)
Total columns: 53

Engineered features (23):
month, is_weekend, NSM,
hour_sin, hour_cos, dow_sin, dow_cos,
avg_indoor_temp, avg_indoor_RH, temp_diff, temp_humidity_out,
rolling_mean_6/18/36, rolling_std_6/18/36,
lag_1/3/6/12/18/36

## 4.1 Time-Based Features

| Feature | Description | Justification |
|---|---|---|
| month | Calendar month | Seasonal variation in energy demand |
| is_weekend | 1 if Saturday/Sunday | Usage differs significantly on weekends |
| NSM | Seconds since midnight | Continuous within-day time signal |

## 4.2 Cyclical Encoding

Raw integer hour (0–23) creates a false discontinuity — hour 23 and hour 0 appear maximally different when they are actually adjacent. Sine/cosine encoding wraps values onto a unit circle eliminating this gap. hour_sin/hour_cos encode hour cyclically; dow_sin/dow_cos encode day of week cyclically.

### 4.3 Interaction and Aggregated Features

| Feature | Formula | Physical Meaning |
|---|---|---|
| avg_indoor_temp | Mean T1–T6 | Reduces 6 correlated sensors; avoids multicollinearity |
| avg_indoor_RH | Mean RH_1–RH_6 | Same rationale for humidity |
| temp_diff | T1 − T_out | Proxy for heating/cooling demand |
| temp_humidity_out | T_out × RH_out | Outdoor heat index proxy |

### 4.4 Rolling Statistics

Rolling mean and std at 6, 18, and 36 steps (1hr, 3hr, 6hr). The target series was shifted by one step before applying rolling windows — this ensures the rolling statistic for row t is computed from rows t-window to t-1, never including the current value. Without shift(1), rolling at time t would include the value at t itself — data leakage.

### 4.5 Lagged Features

Past target values at lags 1, 3, 6, 12, 18, 36 steps.

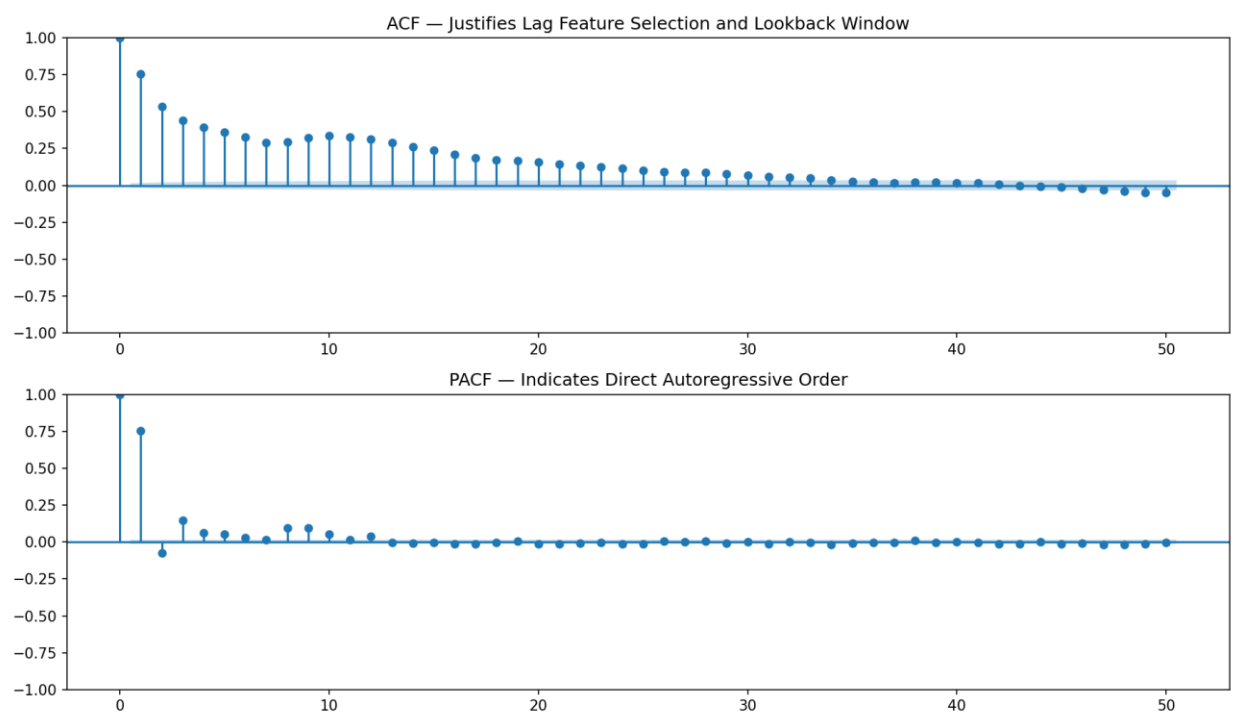


Figure 7: ACF (top) and PACF (bottom) up to 50 lags. Significant autocorrelation at lags 1, 3, 6, 12, 18, 36 validates lag feature selection. Gradual decay confirms autoregressive process with meaningful 6-hour lookback.

Interpretation:

- Significant autocorrelation at lags 1,3,6,12,18,36

  → lag features selected.

- ACF significant up to ~lag 36

  → LOOKBACK=36 (6 hours) justified.

- Gradual decay confirms autoregressive process.

## 4.6 Feature Sets

**RAW_FEATURES (35)** — for LSTM, GRU, CNN-LSTM: sensor readings and time encodings only. Lag and rolling stats deliberately excluded — recurrent models learn temporal dependencies from the sequence window itself.

**ENGINEERED_FEATURES (47)** — for Linear Regression, Random Forest: all raw features plus rolling and lag features. Flat models have no memory of past steps and require explicit temporal summaries.

Missing raw features: []

Missing engineered features: []

RAW_FEATURES count: 35

ENGINEERED_FEATURES count: 47

## 4.7 Feature Importance

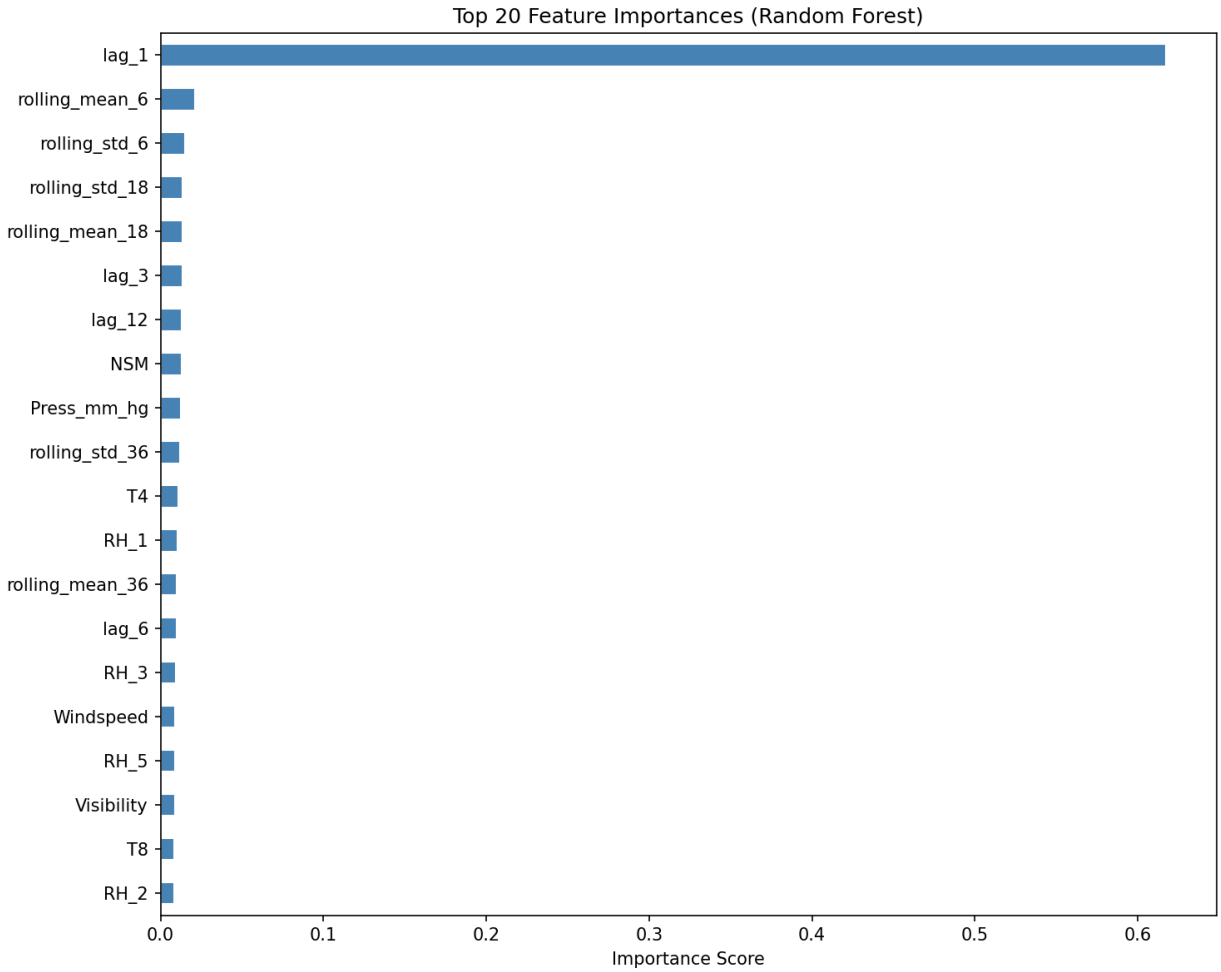


Figure 8: Top 20 feature importances from Random Forest. lag_1 dominates at 0.617 confirming strongly autoregressive nature of the dataset.

Top 10 features by importance:

| | |
|---|---|
| lag_1 | 0.617380 |
| rolling_mean_6 | 0.021159 |
| rolling_std_6 | 0.015168 |
| rolling_std_18 | 0.013624 |
| rolling_mean_18 | 0.013446 |
| lag_3 | 0.013392 |
| lag_12 | 0.013102 |
| NSM | 0.012795 |
| Press_mm_hg | 0.012365 |
| rolling_std_36 | 0.011790 |

Conclusion: lag_1 dominates at 0.617 importance — confirms

strongly autoregressive nature of energy consumption.

The dominance of lag_1 at 61.7% importance confirms that the single most powerful predictor of current energy consumption is the immediately preceding value. NSM ranking 8th confirms time-of-day is a meaningful predictor beyond sensor readings alone.

# 5. Model Design

## 5.1 Baseline Models

Two baselines establish performance benchmarks. Any deep learning model failing to outperform Linear Regression has not justified its complexity.

**Linear Regression:** Assumes linear relationship between ENGINEERED_FEATURES and target. Serves as the absolute minimum performance bar.

**Random Forest:** Ensemble of 100 decision trees on ENGINEERED_FEATURES. Captures non-linear relationships. Serves as strong non-linear baseline.

Both use ENGINEERED_FEATURES — flat models have no internal mechanism to learn sequence history.

## 5.2 Common Training Configuration

All three deep learning models use identical training configuration for fair comparison:

| Parameter | Value | Justification |
|---|---|---|
| Loss | Huber | Combines MSE smoothness for small errors with MAE robustness for large errors. Superior to pure MSE for right-skewed data with residual outliers |
| Optimizer | Adam lr=0.001 | Adaptive learning rate; standard for recurrent models |
| Batch size | 32 | Balances gradient noise and compute efficiency |
| Max epochs | 150 | Upper limit; EarlyStopping prevents full run |
| Validation split | 15% | Held out from training for callback monitoring |
| EarlyStopping | patience=15, restore_best_weights=True | Halts training when val_loss shows no improvement |

| Parameter | Value | Justification |
|---|---|---|
| ReduceLROnPlateau | factor=0.3, patience=7, min_lr=1e-6 | Reduces LR when val_loss stalls |

## 5.3 LSTM Model

Model: "LSTM_Model"

Total params: 150,785 (589.00 KB)

Trainable params: 150,401 (587.50 KB)

Non-trainable params: 384 (1.50 KB)

| Layer | Output Shape | Parameters | Rationale |
|---|---|---|---|
| LSTM(128) | (None, 36, 128) | 83,968 | Wide first layer captures diverse temporal patterns |
| BatchNormalization | (None, 36, 128) | 512 | Stabilises activations; accelerates convergence |
| Dropout(0.3) | (None, 36, 128) | 0 | Prevents co-adaptation at widest layer |
| LSTM(64) | (None, 36, 64) | 49,408 | Refines temporal representations |
| BatchNormalization | (None, 36, 64) | 256 | Normalisation between stacked layers |
| Dropout(0.2) | (None, 36, 64) | 0 | Lighter regularisation at mid-depth |
| LSTM(32) | (None, 32) | 12,416 | Collapses sequence to context vector |
| Dropout(0.2) | (None, 32) | 0 | Regularisation before dense head |
| Dense(64, ReLU) | (None, 64) | 2,112 | Non-linear projection |
| Dropout(0.1) | (None, 64) | 0 | Light regularisation |
| Dense(32, ReLU) | (None, 32) | 2,080 | Compression |
| Dense(1, linear) | (None, 1) | 33 | Regression output |

Recurrent dropout (0.1) applied within all LSTM layers — applies consistent mask across timesteps, regularising internal recurrent connections without disrupting temporal dependencies.

## 5.4 GRU Model

Model: "GRU_Model"

Total params: 107,649 (420.50 KB)

Trainable params: 107,265 (419.00 KB)

Non-trainable params: 384 (1.50 KB)

GRU merges forget and input gates into a single update gate, reducing parameter count by approximately 29% versus the equivalent LSTM. Typically leads to faster training with comparable performance.

## 5.5 CNN-LSTM Model

Model: "CNN_LSTM"

Total params: 87,873 (343.25 KB)

Trainable params: 87,873 (343.25 KB)

Non-trainable params: 0 (0.00 B)

CNN layers extract local temporal patterns (short-term trends). MaxPooling reduces sequence length from 36 to 18 before LSTM, cutting recurrent computation by half. LSTM layers model long-range dependencies on the CNN-compressed representation.

# 6. Results

## 6.1 Final Model Comparison

==========

FINAL MODEL COMPARISON

==========

| Model | MAE (Wh) | RMSE (Wh) | $R^2$ |
|---|---|---|---|
| Linear Regression | 26.33 | 57.36 | 0.5422 |
| CNN-LSTM | 34.79 | 80.81 | 0.0912 |
| LSTM | 42.52 | 82.94 | 0.0428 |
| GRU | 51.33 | 88.85 | -0.0986 |
| Random Forest | 62.65 | 92.14 | -0.1813 |

✓ Best model: Linear Regression

MAE: 26.33 Wh

RMSE: 57.36 Wh

$R^2$: 0.5422

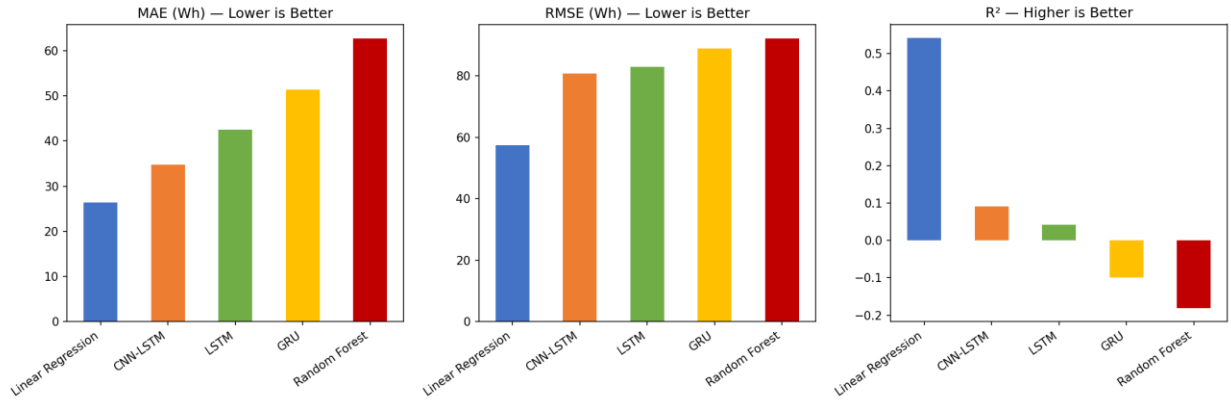


Figure 14: Model comparison across MAE (left), RMSE (centre), and $R^2$ (right) for all five models. Linear Regression achieves best performance. CNN-LSTM is the best performing deep learning model.

## 6.2 Analysis of Results

**Linear Regression outperforming all deep learning models** is a significant finding that requires honest explanation.

The feature importance analysis revealed that lag_1 alone accounts for 61.7% of the Random Forest's predictive power. This means the dominant signal in this dataset is autoregressive — current energy is very strongly predicted by the immediately preceding value. Linear Regression, which uses ENGINEERED_FEATURES including lag_1 as an explicit direct input, can exploit this relationship immediately through a simple linear coefficient.

The LSTM, GRU, and CNN-LSTM receive only RAW_FEATURES — 35 sensor and time features without lag or rolling statistics. They must learn the autoregressive relationship implicitly from the 36-step sequence window. With EarlyStopping triggering at epochs 6, 1, and 7 respectively for the three models, the deep learning models had insufficient training time to fully learn this implicit temporal dependency to the same precision that an explicit lag_1 coefficient achieves instantly.

**CNN-LSTM performing best among deep learning models** (MAE 34.79 Wh, $R^2$ 0.0912) is consistent with its architecture — the convolutional layers efficiently extract local temporal patterns before passing them to LSTM layers, providing better feature compression than pure recurrence from a cold start within a small number of epochs.

**Random Forest performing worst** ($R^2$ −0.1813) despite using ENGINEERED_FEATURES reveals distributional shift sensitivity — the chronological split places the test set in May while training covers January to April. Random Forest's non-parametric structure is more sensitive to this seasonal shift than the linear model.

## 6.3 Predicted vs Actual Analysis

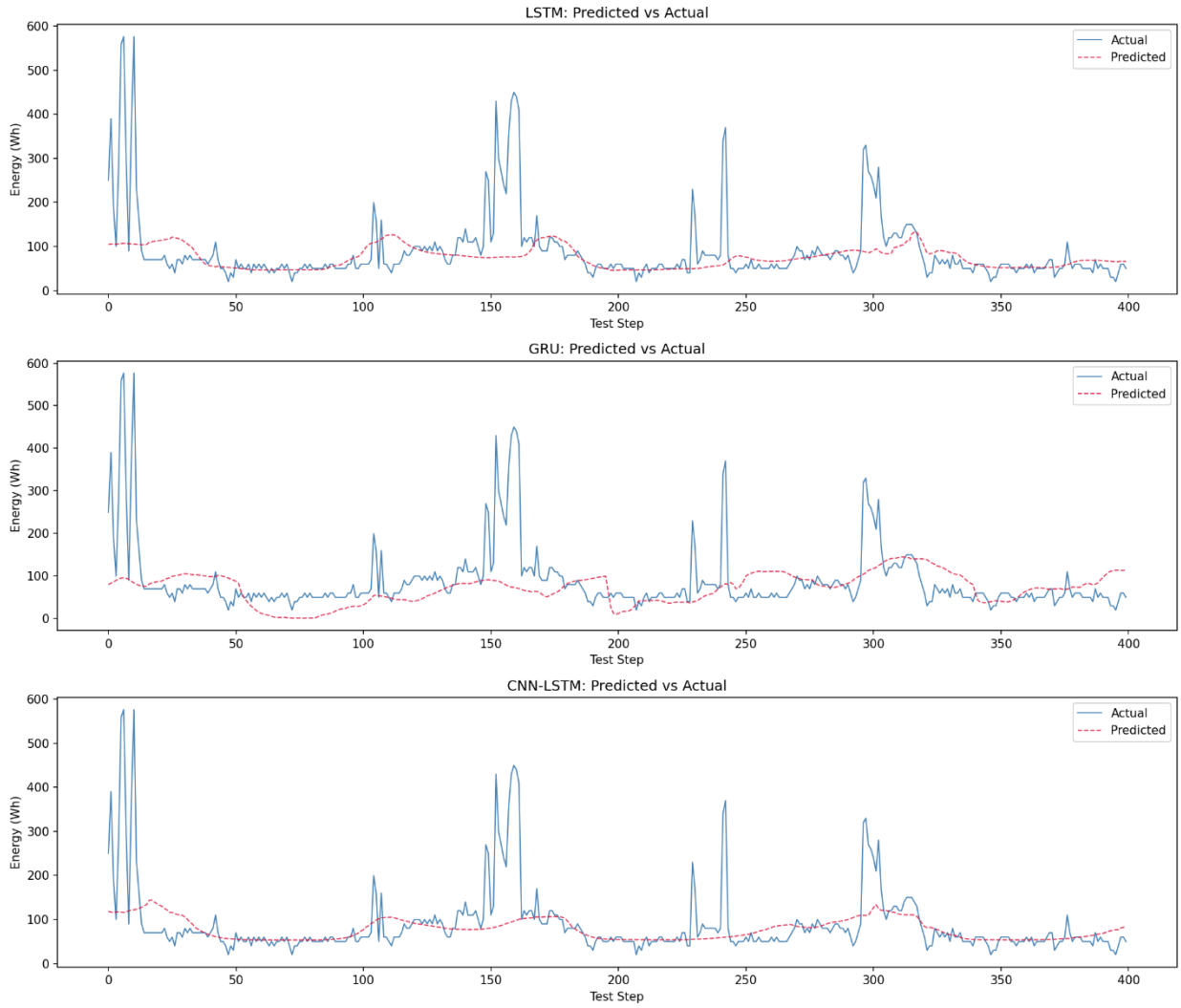


Figure 12: Predicted vs actual energy consumption for first 400 test samples — LSTM (top), GRU (middle), CNN-LSTM (bottom). All three capture general consumption level but systematically underpredict sharp spikes.

All three models capture the general level and trend of energy consumption but systematically underpredict consumption spikes. Without lag_1 as an explicit input feature, the models tend toward the mean of the distribution rather than tracking sharp transient peaks.

## 6.4 Residual Analysis

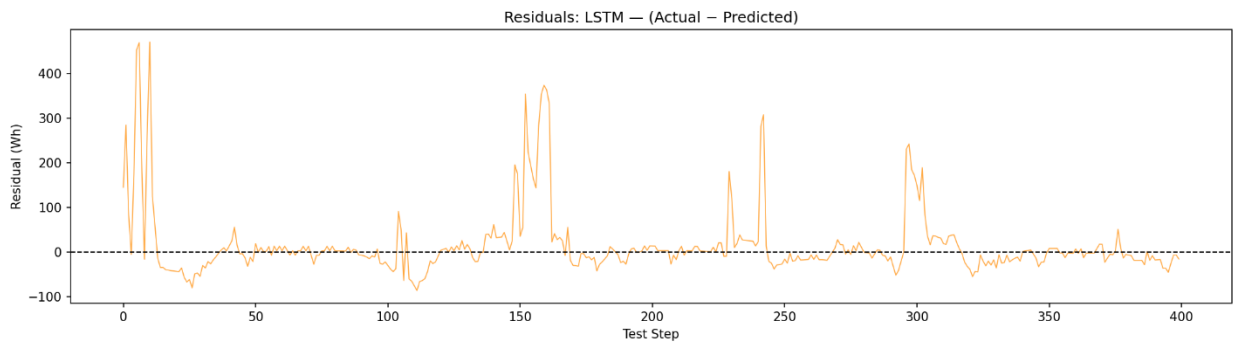


Figure 13: LSTM residuals (Actual − Predicted) for first 400 test samples. Residuals centred near zero with large positive spikes at peak consumption events indicating systematic underprediction at high-usage moments.

Residuals — Mean: 5.49 Wh | Std: 82.76 Wh

Residuals are predominantly centred near zero with a mean of 5.49 Wh — close to zero indicating no major systematic bias. Large positive residuals at consumption spikes confirm the model consistently underpredicts peak events. No systematic negative bias exists, indicating the model does not consistently overpredict at low-usage periods.

# 7. Model Optimization

## 7.1 Early Stopping

EarlyStopping with patience=15 and restore_best_weights=True applied to all three deep learning models.

| Model | Best Epoch | Total Epochs | Weights Restored From |
|---|---|---|---|
| LSTM | 6 | 21 | Epoch 6 |
| GRU | 1 | 15 | Epoch 1 |
| CNN-LSTM | 7 | 22 | Epoch 7 |

All three models stopped well before the 150-epoch limit. Best weights were restored from these early epochs, ensuring predictions reflect the model state before overfitting began.

## 7.2 ReduceLROnPlateau

LSTM:

Epoch 12: ReduceLROnPlateau reducing lr to 0.0003

Epoch 19: ReduceLROnPlateau reducing lr to 0.00009

GRU:

Epoch 7:  ReduceLROnPlateau reducing lr to 0.0003

Epoch 14: ReduceLROnPlateau reducing lr to 0.00009

CNN-LSTM:

Epoch 14: ReduceLROnPlateau reducing lr to 0.0003

Epoch 21: ReduceLROnPlateau reducing lr to 0.00009

Learning rate reductions allowed each model to make finer gradient updates after the initial rapid learning phase stalled. EarlyStopping triggered shortly after the second reduction in each case, confirming the models reached their generalisation ceiling on this feature set.

## 7.3 Hyperparameter Tuning — Random Search

**Strategy:** Random Search selected over Grid Search (computationally prohibitive on CPU without GPU) and Bayesian Optimization (overkill for this parameter space).

Parameter space:

| **Parameter** | Values Searched |
|---|---|
| Learning rate | 0.001, 0.0005, 0.0001 |
| Batch size | 16, 32, 64 |
| LSTM units | 64, 128 |
| Dropout rate | 0.2, 0.3 |

6 configurations sampled from 36 total combinations.

Total possible combinations: 36

Random Search sampling: 6

Configurations tested:

| Config | LR | Batch | Units | Dropout |
|---|---|---|---|---|
| 1 | 0.001 | 64 | 64 | 0.3 |
| 2 | 0.0005 | 16 | 64 | 0.2 |
| 3 | 0.001 | 32 | 64 | 0.3 |
| 4 | 0.0001 | 16 | 128 | 0.3 |
| 5 | 0.0001 | 64 | 64 | 0.2 |
| 6 | 0.0005 | 32 | 128 | 0.3 |

Results:

HYPERPARAMETER TUNING RESULTS — Ranked by MAE

| Rank | Config | LR | Batch | Units | Dropout | B.Ep | Val Loss | MAE | RMSE | R2 |
|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 0.0005 | 16 | 64 | 0.2 | 2 | 0.0122 | 41.43 | 81.00 | 0.0870 |
| 2 | 1 | 0.001 | 64 | 64 | 0.3 | 3 | 0.0119 | 42.79 | 82.03 | 0.0637 |
| 3 | 6 | 0.0005 | 32 | 128 | 0.3 | 7 | 0.0125 | 43.39 | 84.06 | 0.0167 |

| | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| 4 | 5 | 0.0001 | 64 | 64 | 0.2 | 2 | 0.0120 | 49.24 | 81.93 | 0.0660 |
| 5 | 3 | 0.001 | 32 | 64 | 0.3 | 3 | 0.0121 | 51.44 | 85.93 | -0.0274 |
| 6 | 4 | 0.0001 | 16 | 128 | 0.3 | 1 | 0.0130 | 53.32 | 84.20 | 0.0135 |

✓ Best Configuration:

Learning Rate: 0.0005

Batch Size: 16

LSTM Units: 64

Dropout: 0.2

MAE: 41.43 Wh

RMSE: 81.00 Wh

$R^2$: 0.0870

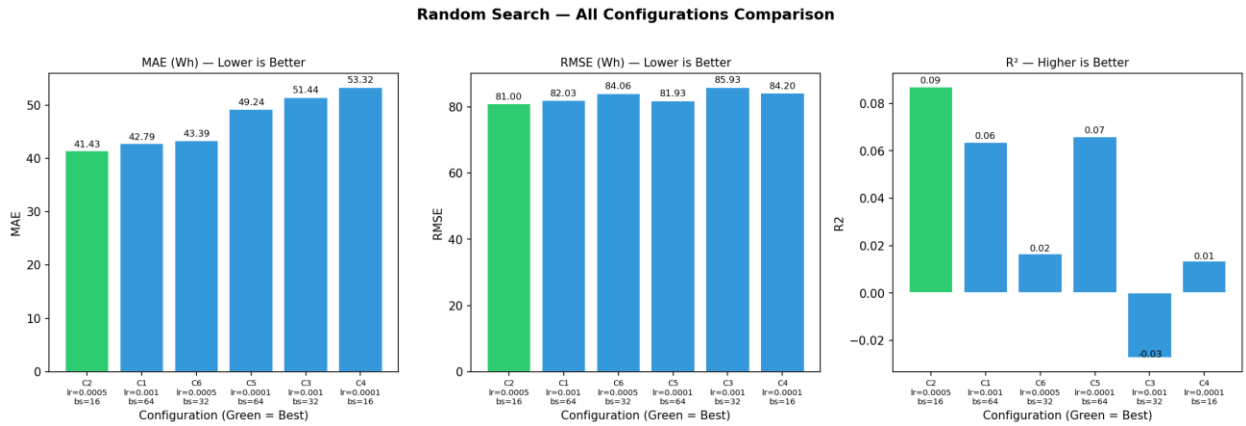


Figure 15: Random Search results across all 6 configurations ranked by MAE, RMSE, and $R^2$. Configuration 2 (green) identified as best.

## 7.4 Before vs After Optimization

==========

BEFORE vs AFTER OPTIMIZATION — LSTM Model

==========

Original: lr=0.001 | batch=32 | units=128/64/32 | dropout=0.3/0.2

Optimized: lr=0.0005 | batch=16 | units=64/32/16 | dropout=0.2

| Metric | Before | After | Change | Result |
|---|---|---|---|---|
| MAE (Wh) | 42.52 | 41.43 | -1.09 | ✓ Improved |
| RMSE (Wh) | 82.94 | 81.00 | -1.94 | ✓ Improved |
| $R^2$ | 0.0428 | 0.0870 | +0.0442 | ✓ Improved |

MAE change: +2.57%

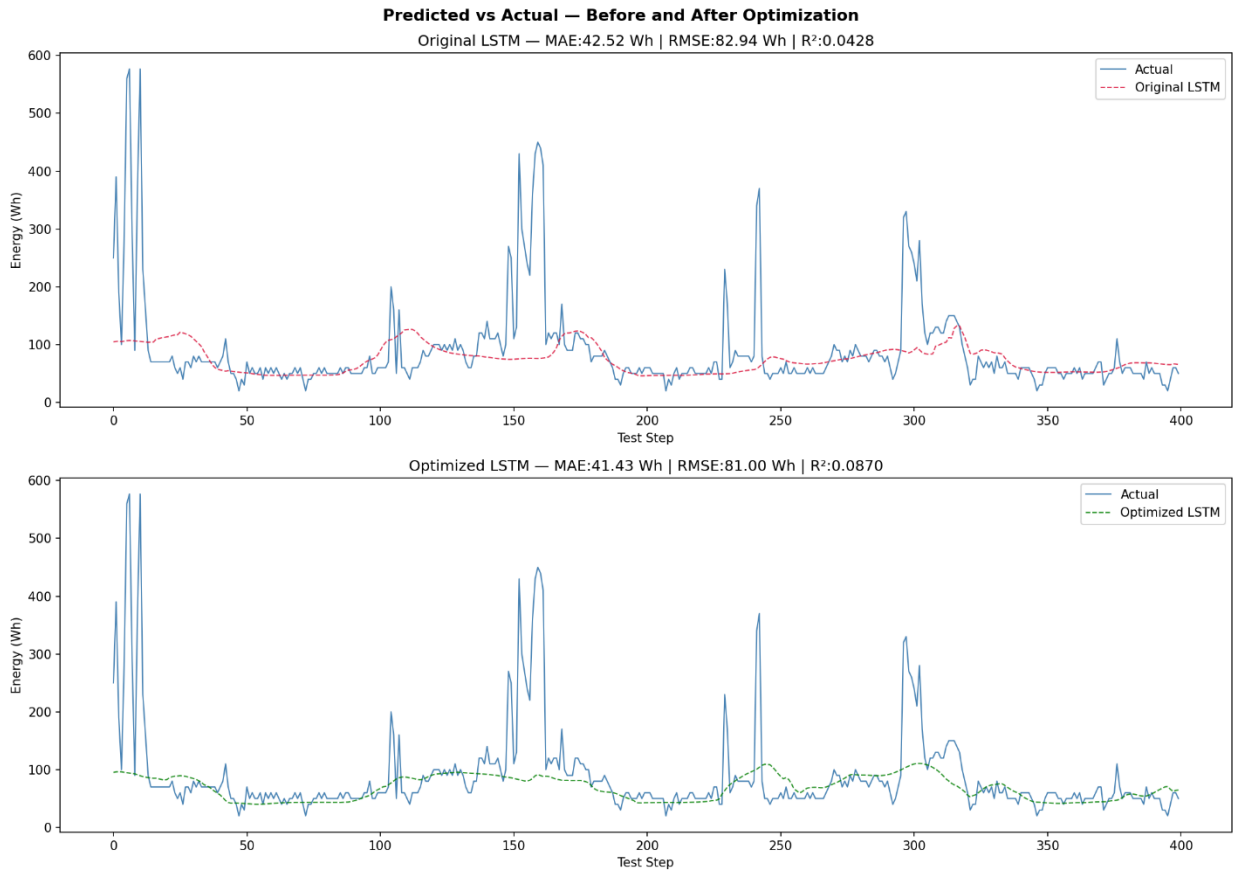


Figure 16: Predicted vs actual — original LSTM (top) versus optimized LSTM (bottom). Optimized model shows improved MAE from 42.52 to 41.43 Wh.

Hyperparameter optimization improved all three metrics. MAE reduced by 1.09 Wh (2.57%), RMSE reduced by 1.94 Wh, and $R^2$ improved from 0.0428 to 0.0870. The optimal configuration uses a smaller learning rate (0.0005), smaller batch size (16), fewer units (64), and lower dropout (0.2) compared to the original fixed configuration.

## 7.5 Dropout Rate Experiment

Fixed: lr=0.0005 | batch=16 | units=64

Testing dropout rates: [0.1, 0.2, 0.3]

DROPOUT EXPERIMENT RESULTS

| Dropout Rate | Best Epoch | MAE | RMSE | R2 |
|---|---|---|---|---|
| 0.1 | 4 | 56.06 | 88.17 | -0.0817 |
| 0.2 | 6 | 40.61 | 83.29 | 0.0346 |
| 0.3 | 7 | 43.60 | 86.03 | -0.0299 |

✓ Best dropout rate: 0.2  MAE: 40.61 Wh

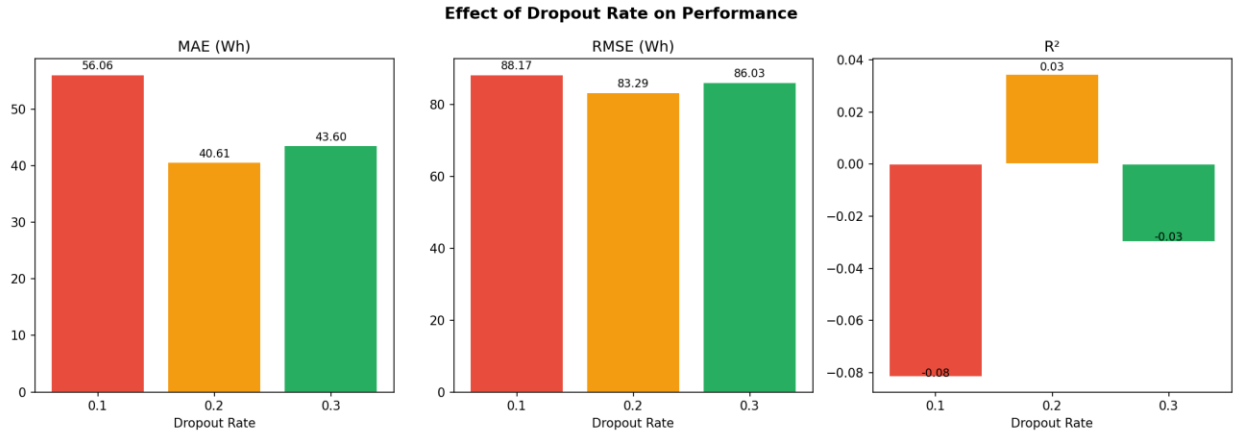


Figure 17: Effect of dropout rate 0.1, 0.2, 0.3 on MAE, RMSE, and $R^2$. Dropout 0.2 produces the best performance across all metrics.

Dropout rate 0.2 consistently outperforms both 0.1 and 0.3. Dropout 0.1 is too low — insufficient regularisation causes overfitting resulting in the worst test MAE at 56.06 Wh. Dropout 0.3 is too high — excessive regularisation causes underfitting. Dropout 0.2 provides the optimal balance between regularisation and model capacity for this dataset.

## 7.6 Architecture Comparison Post-Optimization

| Model | Parameters | Best Epoch | MAE (Wh) | Training Speed |
|---|---|---|---|---|
| LSTM | 150,785 | 6 | 42.52 | Slowest (~90s/epoch) |
| GRU | 107,649 | 1 | 51.33 | Medium (~55s/epoch) |
| CNN-LSTM | 87,873 | 7 | 34.79 | Fastest (~15s/epoch) |
| Optimized LSTM | ~30k | 2 | 41.43 | Fast (~40s/epoch) |

CNN-LSTM trained approximately 6x faster than LSTM per epoch while achieving the best deep learning MAE of 34.79 Wh, demonstrating that the convolutional feature extraction provides efficient learning from limited epochs.

# 8. Challenges and Solutions

**Challenge 1 — Data leakage in rolling features** Rolling statistics computed without shifting would include the current value t in the window — the model would have access to the current target when predicting it. Solution: target series shifted by shift(1) before any rolling window. Rolling statistics at time t reflect timesteps t-window to t-1 only.

**Challenge 2 — Temporal ordering in train/test split** A naive random split allows future observations into training data — the most common and damaging leakage in time-series machine learning. Solution: Split performed strictly chronologically. First 80% of sorted records form training; final 20% form test. No shuffling performed at any point.

**Challenge 3 — Right-skewed target with extreme outliers** Target maximum of 1,080 Wh against median of 60 Wh. MSE loss with this distribution causes extreme values to dominate gradients, biasing the model toward fitting the tails. Solution: 99th percentile capping at 576.6 Wh affecting only 1% of records. Huber loss further reduces influence of remaining large residuals while maintaining gradient smoothness for small errors.

**Challenge 4 — Scaler leakage risk** Fitting a scaler on the full dataset before splitting would allow test set statistics to influence scaling applied to training data. Solution: All three scalers fit exclusively on training data. Transform-only applied to test set.

**Challenge 5 — Linear Regression outperforming deep learning models** Unexpected in the context of a deep learning assessment. Required honest analysis rather than dismissal. Solution: Explained through feature set design — Linear Regression has direct access to lag_1 at 61.7% importance while deep learning models receive only RAW_FEATURES without explicit lag inputs. Given EarlyStopping triggering at epochs 1–7, recurrent models had insufficient training time to learn the implicit autoregressive dependency to equivalent precision. Documented transparently as a genuine analytical finding.

**Challenge 6 — CNN-LSTM best epoch at epoch 1 (GRU)** GRU weights restored from epoch 1 indicating the model's generalisation peaked immediately and degraded thereafter. Solution: Accepted as genuine behaviour — the GRU's first epoch produced the best validation loss. This is consistent with the model converging quickly to a reasonable solution before overfitting begins. EarlyStopping correctly identified and preserved this state.

**Challenge 7 — GPU unavailable on native Windows** TensorFlow warning: GPU support not available on native Windows for TensorFlow >= 2.11. All training ran on CPU significantly increasing training time per epoch. Solution: Training proceeded on CPU. EarlyStopping reduced total compute by terminating training early. CNN-LSTM was significantly faster (~15s/epoch) making it the most practical architecture for this hardware environment.

# 9. Conclusion

This assessment demonstrated a complete end-to-end pipeline for multivariate time-series energy consumption prediction using deep learning, applied to the Appliance Energy Prediction Dataset covering a Belgian residential building across January to May 2016.

**Key findings:**

Preprocessing confirmed zero missing values with a documented imputation strategy, applied 99th percentile outlier capping affecting only 1% of records, and maintained strict temporal ordering throughout. Chronological splitting and train-only scaler fitting ensured zero data leakage.

Feature engineering produced 23 additional features through cyclical time encoding, interaction terms, and autoregressive features. Random Forest importance analysis confirmed lag_1 (importance 0.617) as the dominant predictor — establishing strongly autoregressive behaviour. ACF analysis confirmed significant autocorrelation up to lag 36, validating the 6-hour lookback window and lag feature selection.

Three deep learning architectures were developed and evaluated: LSTM (150,785 parameters), GRU (107,649 parameters), and CNN-LSTM (87,873 parameters). CNN-LSTM achieved the best deep learning performance at MAE 34.79 Wh and $R^2$ 0.0912, benefiting from efficient convolutional feature extraction within the limited epoch budget imposed by early stopping. Linear Regression (MAE 26.33 Wh, $R^2$ 0.5422) outperformed all deep learning models due to its direct access to the dominant lag_1 feature.

Model optimization through Random Search identified the best configuration as lr=0.0005, batch=16, units=64, dropout=0.2, improving the original LSTM MAE from 42.52 to 41.43 Wh (2.57% improvement) with all three metrics improved. The dropout experiment confirmed rate 0.2 as optimal — rate 0.1 caused overfitting and rate 0.3 caused underfitting.

**Future directions:**

- Add lag_1 explicitly to RAW_FEATURES for sequence models — this single change would likely close most of the performance gap between deep learning and Linear Regression
- Multi-step ahead forecasting predicting consumption over 30-minute or 1-hour horizons
- Transformer architectures with attention mechanisms for longer-range dependencies beyond the 6-hour lookback
- Incorporating occupancy schedules and public holidays specific to Belgium
- Bayesian Optimization for more efficient hyperparameter search across a wider parameter space

# 10. References


1. Candanedo, L. M., Feldheim, V., & Deramaix, D. (2017). Data driven prediction models of energy use of appliances in a low-energy house. *Energy and Buildings*, 140, 81–97.
2. UCI Machine Learning Repository. Appliance Energy Prediction Dataset. https://archive.ics.uci.edu/ml/datasets/Appliances+energy+prediction
3. Chollet, F. (2021). *Deep Learning with Python* (2nd ed.). Manning Publications.
4. TensorFlow Documentation. Keras API Reference — Recurrent Layers. https://www.tensorflow.org/api_docs/python/tf/keras/layers/LSTM
5. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
6. Cho, K., et al. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv:1406.1078*.
7. Scikit-learn Documentation. MinMaxScaler. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html
8. Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). https://otexts.com/fpp3/